gt

— DESIGN REFINEMENT, V0.1

A BEAUTIFUL UNIX INSTRUMENT

Polishing gt for daily use.

Hierarchy, color, density — refined cell by cell. Plus an inline diff sketch and a stash that earns its keystroke.

Repo	github.com/pandr/gt
Stack	Go · bubbletea · lipgloss
Aesthetic	Modern TUI, ASCII-flavored
Output	Implementation ideas

Index `12 SLIDES`

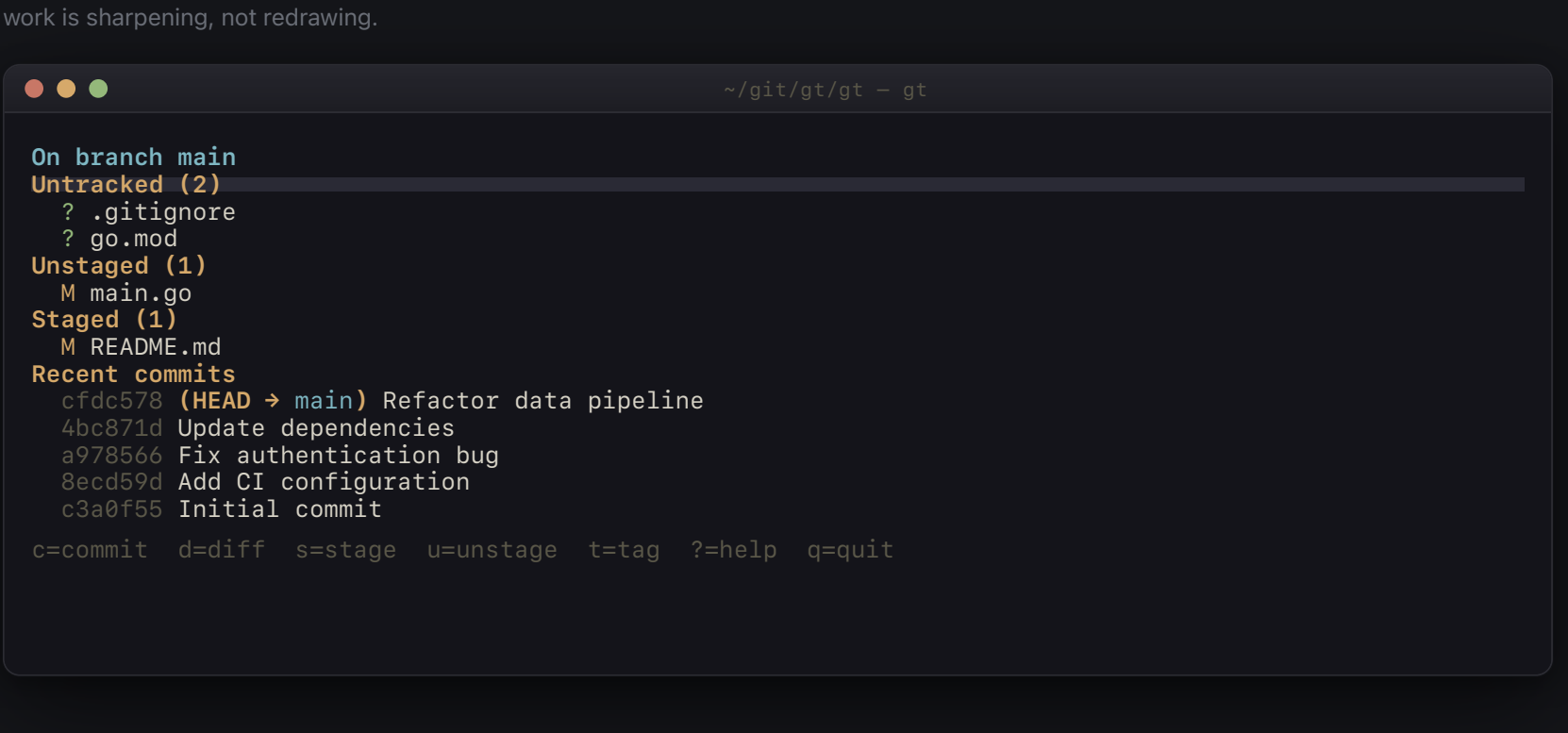
Three movements: tighten the overview, propose an inline diff, sketch a stash that fits gt's grammar. Each idea is sized to be implementable; nothing here breaks the existing model.

01	The current overview	baseline	What gt looks like today, read carefully.
02	Palette & glyph kit	design system	Two accents, three states. The token list.
03	Overview, refined	primary	Hierarchy via weight + box-drawing. Same density.
04	Before / after	comparison	Side-by-side. The same fixture, two treatments.
05	Density variants	explore	Tight · default · breathing.
06	Cursor & tag micro-decisions	explore	Three takes on the cursor row.
07	Inline diff: unified	new	Stay in gt; preserve the cell grid.
08	Inline diff: split	new	Two columns, one row of context per change.
09	Diff: design rationale	why	Where inline beats `less`, and where it doesn't.
10	Stash, mutt-style	new feature	A fourth section, two new keys, no new modes.
11	Stash: pop & show	new feature	What happens when you press p.
12	Implementation notes	handoff	For Claude Code: the diff to apply.

Read in order or jump — each slide stands alone. j/k · ←/→

The current overview `BASELINE`

Reading what's actually there. The screen is already mutt-disciplined — dense, scannable, monochrome with three accent slots. The work is sharpening, not redrawing.



```
~/git/gt/gt — gt

On branch main
Untracked (2)
  ? .gitignore
  ? go.mod
Unstaged (1)
  M main.go
Staged (1)
  M README.md
Recent commits
  cfdc578 (HEAD → main) Refactor data pipeline
  4bc871d Update dependencies
  a978566 Fix authentication bug
  8ecd59d Add CI configuration
  c3a0f55 Initial commit
c=commit  d=diff  s=stage  u=unstage  t=tag  ?=help  q=quit
```

WHAT'S ALREADY RIGHT

- Section headers bold & warm, file rows neutral — correct hierarchy
- Section counts in parens — mutt pattern, instantly readable
- X/Y indicators (?/M/A/D) carry meaning by glyph, not just color
- Context-aware hint bar at bottom — teaches the keymap

WHAT I WANT TO TIGHTEN

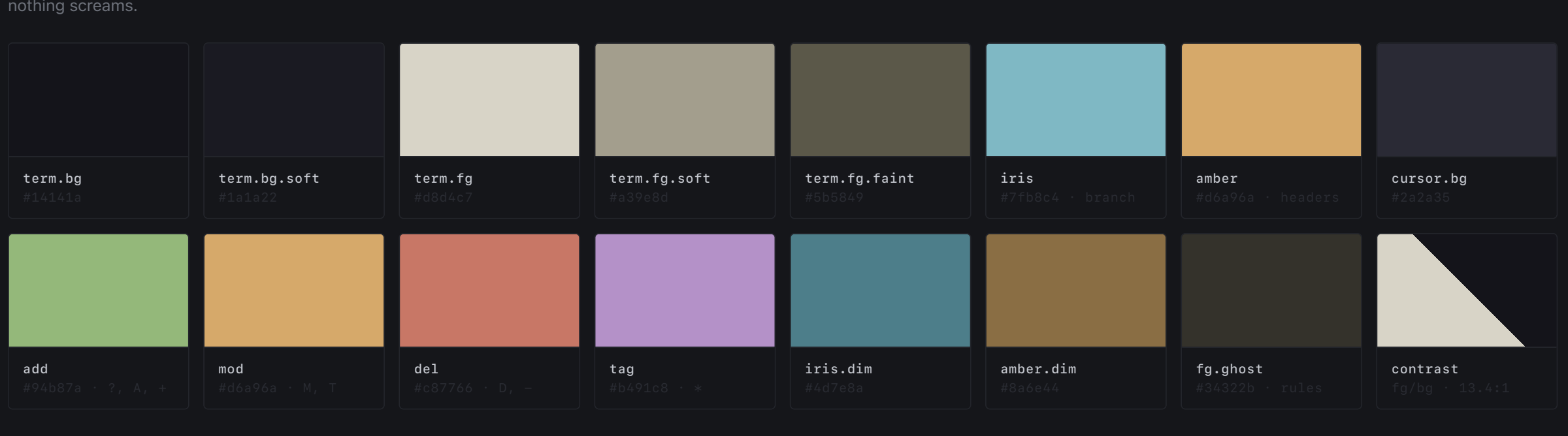
- All four section headers are the *same* amber — the eye can't tell "has work" from "empty"
- Cursor row spans the full terminal width even when content is short — reads as a bar, not a pin
- No visible relationship between sections; they float in the void
- HEAD ref is amber-bold which fights the section headers for attention
- Indicator column has no internal alignment between sections

> Goal: same density, sharper hierarchy, accents that earn their use. Don't make it pretty — make it correct.

🟧 amber · headers, change 🟦 iris · branch, identity 🟩 moss · added 🟥 terracotta · deleted 🟪 mauve · tagged 96 cols · 14px/19px cell

Palette & glyph kit `TOKENS`

Two accents (iris, amber) plus three semantic states (add, mod, del) and one extra (tag). Everything else is luminance. Low chroma; nothing screams.



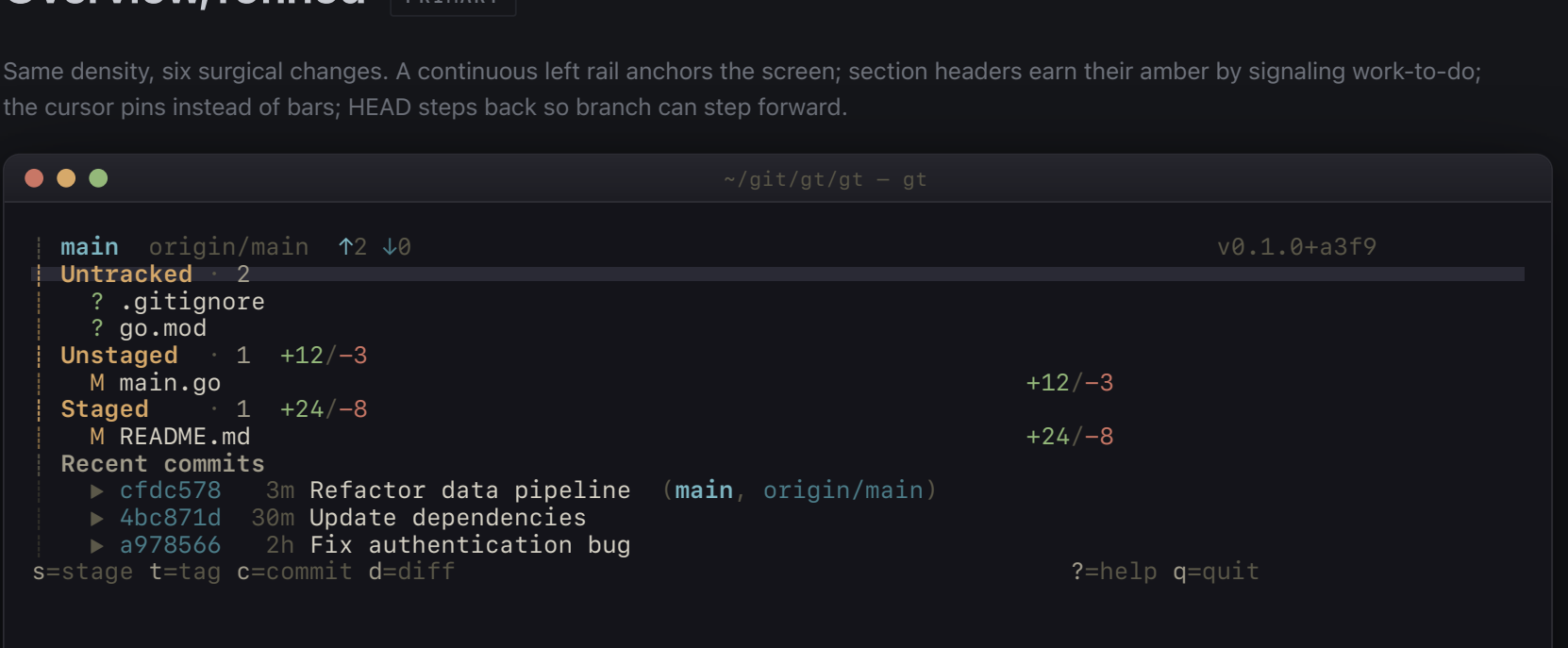
term.bg #14141a	term.bg.soft #1a1a22	term.fg #d8d4c7	term.fg.soft #a39e8d	term.fg.faint #5b5849	iris #7fb8c4 · branch	amber #d6a96a · headers	cursor.bg #2a2a35
add #94b87a · ?, A, +	mod #d6a96a · M, T	del #c87766 · D, −	tag #b491c8 · *	iris.dim #4d7e8a	amber.dim #8a6e44	fg.ghost #34322b · rules	contrast fg/bg · 13.4:1

▶	▼	∗	·	┆	↑2 ↓0
collapsed	expanded	tagged	commit · file	section rule	ahead/behind
Directory or commit, closed.	Open. Pairs with ▶.	Replaces leading column. Mauve.	Bullet for files inside an open commit.	Dotted vertical, gutter use only.	Fixed-width pair next to upstream.

Contrast: fg/bg 13.4:1 · iris/bg 6.1:1 · amber/bg 7.2:1 — all WCAG AA at body size oklch family · constant chroma 0.06

Overview, refined `PRIMARY`

Same density, six surgical changes. A continuous left rail anchors the screen; section headers earn their amber by signaling work-to-do; the cursor pins instead of bars; HEAD steps back so branch can step forward.



```
~/git/gt/gt — gt

┆ main  origin/main  ↑2 ↓0                                      v0.1.0+a3f9
┆ Untracked  · 2
┆   ? .gitignore
┆   ? go.mod
┆ Unstaged   · 1  +12/−3
┆   M main.go                                          +12/−3
┆ Staged     · 1  +24/−8
┆   M README.md                                        +24/−8
┆ Recent commits
┆   ▶ cfdc578   3m Refactor data pipeline  (main, origin/main)
┆   ▶ 4bc871d  30m Update dependencies
┆   ▶ a978566   2h Fix authentication bug
s=stage t=tag c=commit d=diff                                ?=help q=quit
```

SIX REFINEMENTS

- **Left rail.** A 1-cell ┆ column ties the screen together. Bright on sections with work, dimmed when empty.
- **Header rhythm.** Counts as · 2 after the name — lighter than parens, mutt-style.
- **Diff-stat on headers.** `+12 / −3` next to the section name — you see the size of the change before drilling in.
- **Right-aligned per-file stats.** Uses the empty space we already have.
- **"Recent commits" demoted.** Soft fg, not amber — it's reference, not action.
- **Commit row layout.** `▶ sha 3m title (refs)` — age in 4-cell field, refs trail.

WHAT STAYED

- Every keystroke. The model.go contract is unchanged.
- Section order. Untracked → Unstaged → Staged → Working tree → Log.
- X/Y indicators. `? M A D` still carry the same colors.

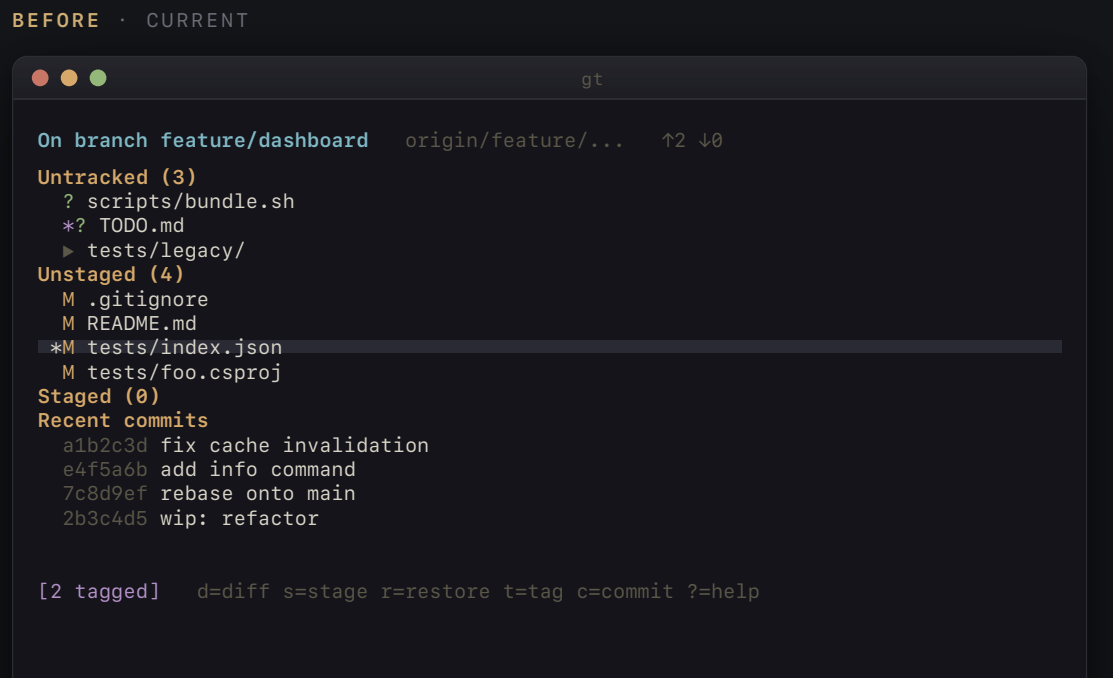
> The rail is doing real work: it's the section attendance light. Bright = there are files here. Ghost = empty section, still navigable. You can read the screen at a glance from across the room.

Cursor pins to content width — doesn't bleed across the terminal. diff vs baseline: 6 changes, 0 new keys

Before & after `COMPARISON`

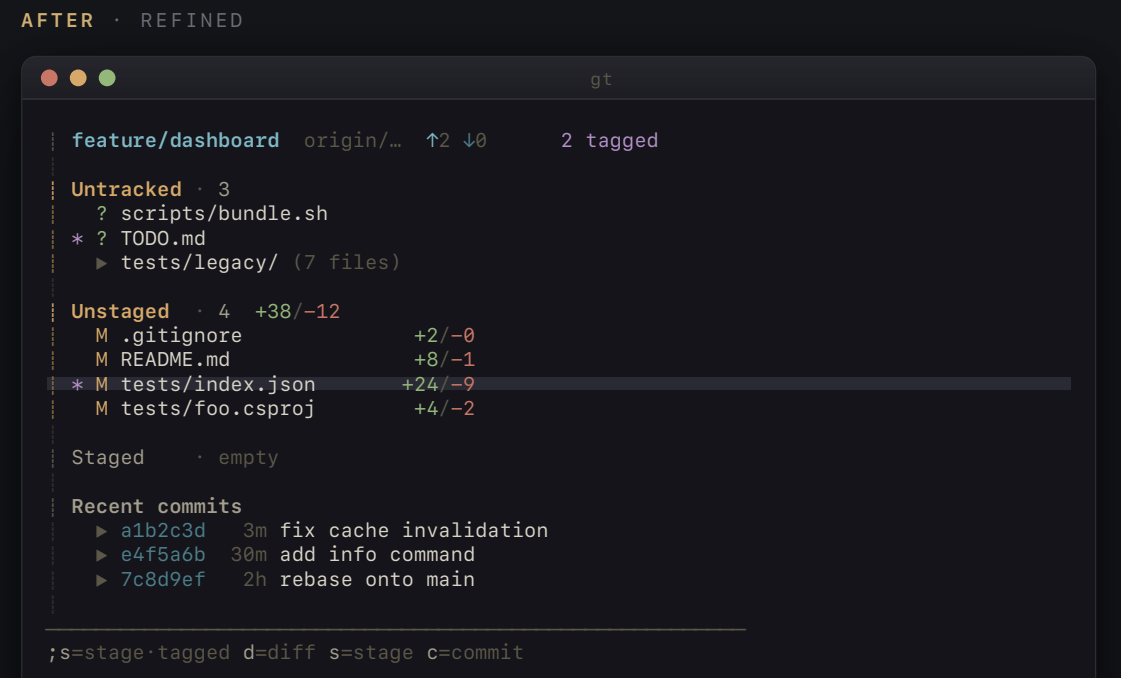
A more loaded fixture: tagged files across sections, working-tree open, expanded commit. Read both screens; the after should require one less saccade.

BEFORE · CURRENT



```
gt

On branch feature/dashboard   origin/feature/...   ↑2 ↓0
Untracked (3)
  ? scripts/bundle.sh
 *? TODO.md
  ▶ tests/legacy/
Unstaged (4)
  M .gitignore
  M README.md
 *M tests/index.json
  M tests/foo.csproj
Staged (0)
Recent commits
  a1b2c3d fix cache invalidation
  e4f5a6b add info command
  7c8d9ef rebase onto main
  2b3c4d5 wip: refactor

[2 tagged]   d=diff s=stage r=restore t=tag c=commit ?=help
```

AFTER · REFINED

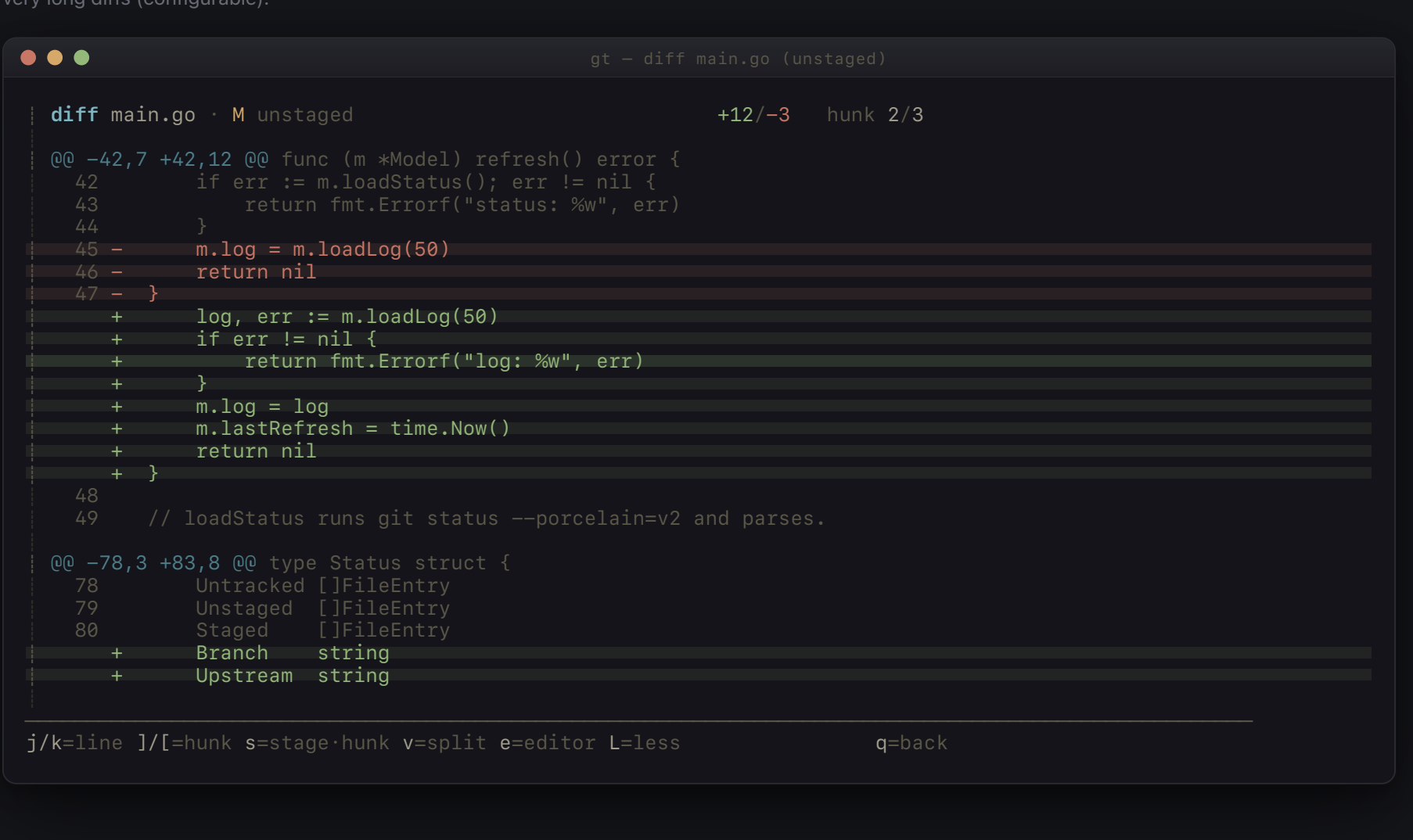


```
gt

┆ feature/dashboard  origin/…  ↑2 ↓0      2 tagged

┆ Untracked  · 3
┆   ? scripts/bundle.sh
┆ * ? TODO.md
┆   ▶ tests/legacy/ (7 files)

┆ Unstaged  · 4  +38/−12
┆   M .gitignore              +2/−0
┆   M README.md               +8/−1
┆ * M tests/index.json       +24/−9
┆   M tests/foo.csproj        +4/−2

┆ Staged    · empty

┆ Recent commits
┆   ▶ a1b2c3d   3m fix cache invalidation
┆   ▶ e4f5a6b  30m add info command
┆   ▶ 7c8d9ef   2h rebase onto main
┆
──────────────────────────────────────────────────
;s=stage·tagged d=diff s=stage c=commit
```

Both screens are exact char-by-char terminal renders. No tricks. see also: 05

Inline diff — unified `NEW`

Press d, the diff opens *in* gt instead of shelling to `less`. Same cell grid, same keys, navigation by hunk. q returns. Falls back to `less` for very long diffs (configurable).



```
gt — diff main.go (unstaged)

┆ diff main.go · M unstaged                          +12/−3   hunk 2/3
┆
┆ @@ -42,7 +42,12 @@ func (m *Model) refresh() error {
┆   42        if err := m.loadStatus(); err != nil {
┆   43            return fmt.Errorf("status: %w", err)
┆   44        }
┆   45 −      m.log = m.loadLog(50)
┆   46 −      return nil
┆   47 −  }
┆      +      log, err := m.loadLog(50)
┆      +      if err != nil {
┆      +          return fmt.Errorf("log: %w", err)
┆      +      }
┆      +      m.log = log
┆      +      m.lastRefresh = time.Now()
┆      +      return nil
┆      +  }
┆   48
┆   49    // loadStatus runs git status --porcelain=v2 and parses.
┆
┆ @@ -78,3 +83,8 @@ type Status struct {
┆   78        Untracked []FileEntry
┆   79        Unstaged  []FileEntry
┆   80        Staged    []FileEntry
┆      +      Branch    string
┆      +      Upstream  string
┆
──────────────────────────────────────────────────────────────────
j/k=line ]/[=hunk s=stage·hunk v=split e=editor L=less             q=back
```

ANATOMY

- **Header line** identifies file + section + diff stat + hunk position. Same rail as overview.
- **Old/new line numbers** as two 4-cell columns — stays aligned even with 4-digit lines.
- **Sign column** (+/−/) is the one piece carrying color.
- **Soft tinted background** on changed rows — 10% alpha; cursor row goes to 18%. Color isn't doing the heavy lifting.
- **Hunk separator** is the existing rail glyph at `fg-ghost` luminance — structural, near-invisible.

NEW KEYS (SMALL ADDITIONS)

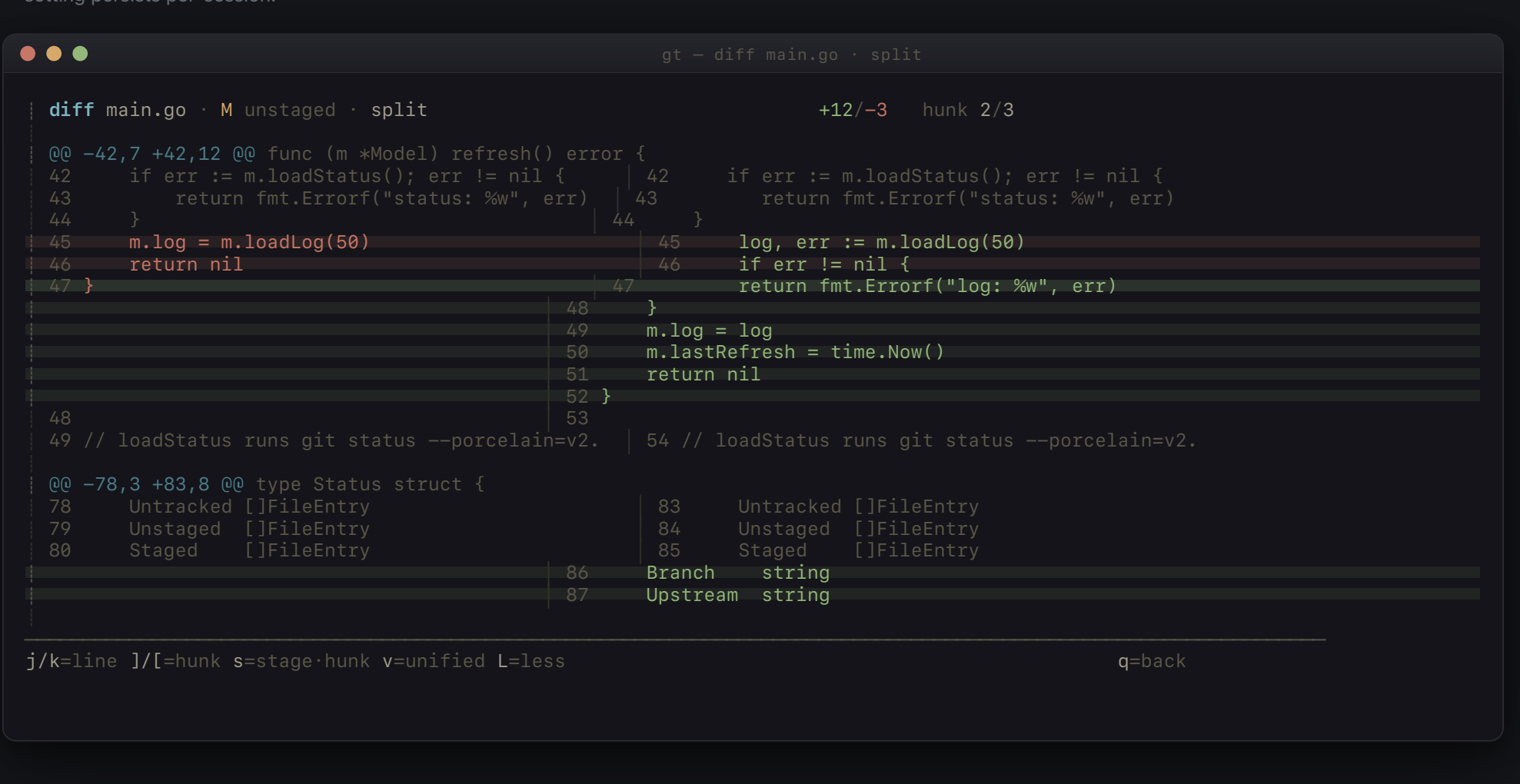
- `]` / `[` jump to next/prev hunk
- `s` here stages just *this hunk* — the long-deferred `git add -p` entry point
- `v` toggles split (next slide)
- `L` hands off to `$PAGER` if you want the full `less` experience

> The diff doesn't replace `less`; it makes the most common diff (one file, a few hunks) feel native. Long diffs still belong in the pager.

Implementation: parse git diff --no-color -U3 output. ~150 lines. fallback threshold: configurable (default 1000 lines)

Inline diff — split `V TOGGLES`

Same hunk, two columns. Old left, new right. The vertical box-drawing rule between them is the only chrome. Press v in unified to switch — setting persists per-session.



```
gt — diff main.go · split

┆ diff main.go · M unstaged · split                                 +12/−3   hunk 2/3
┆
┆ @@ -42,7 +42,12 @@ func (m *Model) refresh() error {
┆ 42     if err := m.loadStatus(); err != nil {     │ 42     if err := m.loadStatus(); err != nil {
┆ 43         return fmt.Errorf("status: %w", err)   │ 43         return fmt.Errorf("status: %w", err)
┆ 44     }                                          │ 44     }
┆ 45     m.log = m.loadLog(50)                      │ 45     log, err := m.loadLog(50)
┆ 46     return nil                                 │ 46     if err != nil {
┆ 47 }                                              │ 47         return fmt.Errorf("log: %w", err)
┆                                                   │ 48     }
┆                                                   │ 49     m.log = log
┆                                                   │ 50     m.lastRefresh = time.Now()
┆                                                   │ 51     return nil
┆                                                   │ 52 }
┆ 48                                                │ 53
┆ 49 // loadStatus runs git status --porcelain=v2.  │ 54 // loadStatus runs git status --porcelain=v2.
┆
┆ @@ -78,3 +83,8 @@ type Status struct {
┆ 78     Untracked []FileEntry                      │ 83     Untracked []FileEntry
┆ 79     Unstaged  []FileEntry                      │ 84     Unstaged  []FileEntry
┆ 80     Staged    []FileEntry                      │ 85     Staged    []FileEntry
┆                                                   │ 86     Branch    string
┆                                                   │ 87     Upstream  string
┆
────────────────────────────────────────────────────────────────────────────────────
j/k=line ]/[=hunk s=stage·hunk v=unified L=less                                q=back
```

WHEN TO PREFER SPLIT

- Refactors where lines move — alignment is much faster to read
- Wide enough terminal (≥120 cols); auto-falls-back to unified below 100
- You're in a code-review mood, not a stage-and-go mood

IMPLEMENTATION NOTE

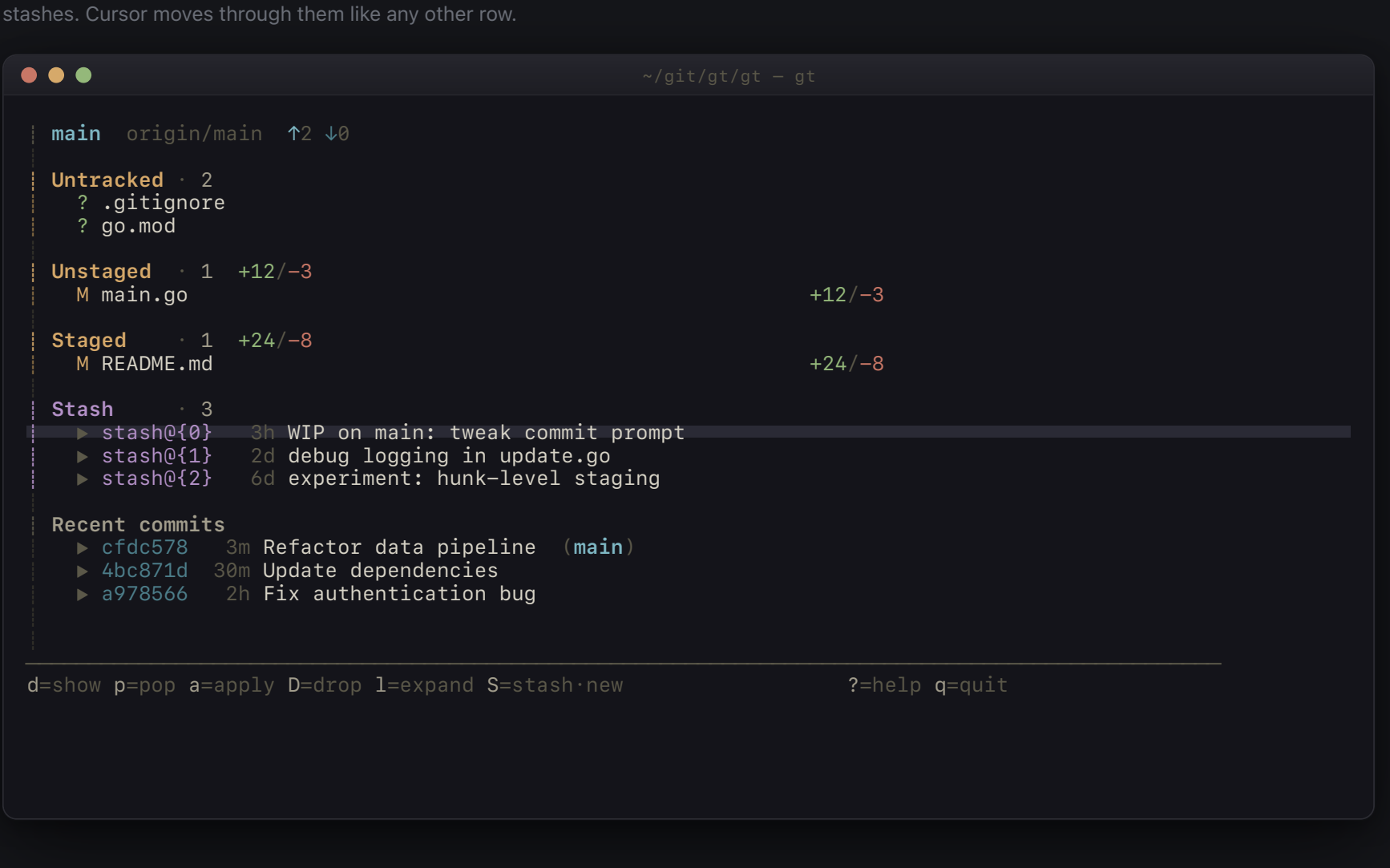
- Server-side line pairing: walk the unified diff, assign each −/+ pair the same row index where possible; bare − or + get an empty cell on the other side.
- The vertical │ rail is at fg-ghost luminance; it disappears when you stop looking at it — structural, not decorative.
- Cursor highlight applies to *both columns* of the same logical row.

> The split is one pixel of CSS apart from the unified. Same model, same parser, same keys. v toggles. That's the whole feature.

Both modes route hunk-stage through the same plumbing. ≥120 cols recommended for split

Stash, mutt-style `NEW FEATURE`

Stash isn't a separate screen — it's a fourth section, sitting between Staged and Working tree. Empty by default; appears when you have stashes. Cursor moves through them like any other row.



```
~/git/gt/gt — gt

┆ main  origin/main  ↑2 ↓0

┆ Untracked  · 2
┆   ? .gitignore
┆   ? go.mod

┆ Unstaged   · 1  +12/−3
┆   M main.go                                    +12/−3

┆ Staged     · 1  +24/−8
┆   M README.md                                  +24/−8

┆ Stash      · 3
┆   ▶ stash@{0}   3h WIP on main: tweak commit prompt
┆   ▶ stash@{1}   2d debug logging in update.go
┆   ▶ stash@{2}   6d experiment: hunk-level staging

┆ Recent commits
┆   ▶ cfdc578   3m Refactor data pipeline  (main)
┆   ▶ 4bc871d  30m Update dependencies
┆   ▶ a978566   2h Fix authentication bug
┆
─────────────────────────────────────────────────────────────
d=show p=pop a=apply D=drop l=expand S=stash·new         ?=help q=quit
```

WHY A SECTION, NOT A SCREEN

- Stash is just *state*, like Untracked and Staged. It belongs in the inventory.
- You see it without asking. The first time you stash you'll find it without reading docs.
- Empty stash = section hidden — same rule as "Staged · empty".

THE GRAMMAR HOLDS

- `l` on a stash row expands its file list (just like a commit)
- `d` diffs the stash (just like a commit)
- `t` tags it — `;d` diffs all tagged stashes back-to-back
- Mauve ┆ rail visually separates "parked work" from active changes

TWO NEW KEYS

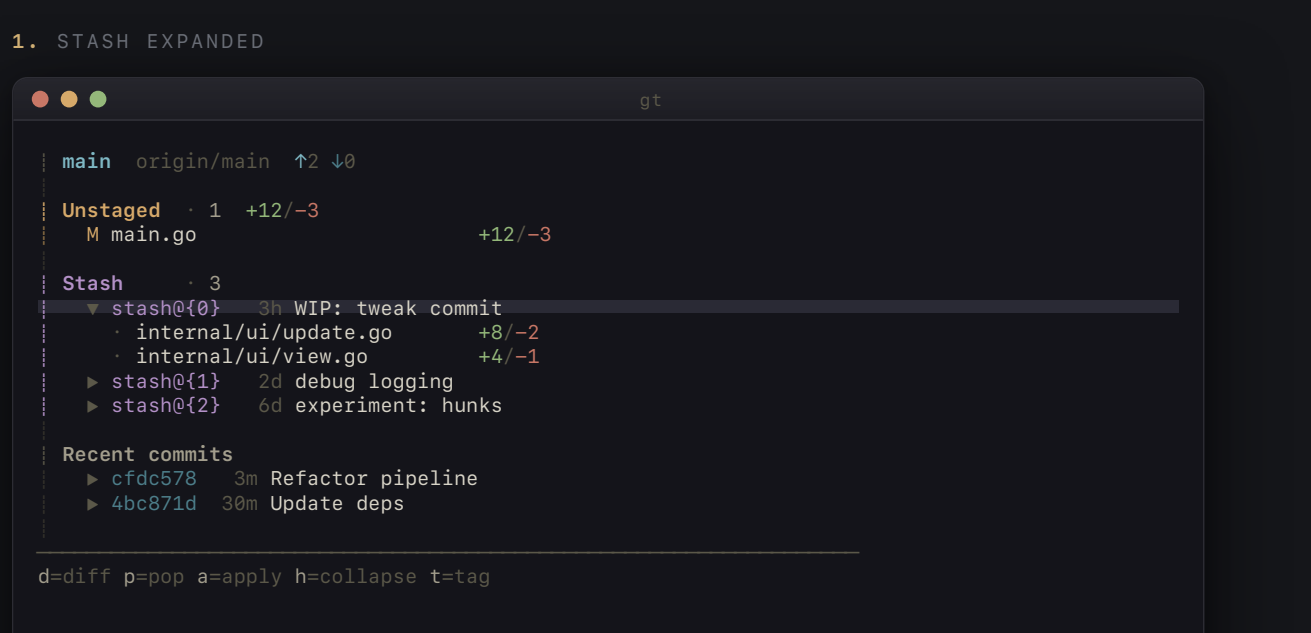
- `S` — *stash all*. Opens a one-line message prompt, same UX as `c`. Empty message = no message.
- `p` — pop. Only valid on a stash row. Asks `y / n` if stash isn't on current branch.
- `a` apply, `D` drop — only on stash rows; out of the way otherwise.

Mauve rail = the same color as the tag glyph — reinforces "set aside" semantics. git stash list / show / pop / apply / drop / push -m

Expanding a stash, popping it `INTERACTION`

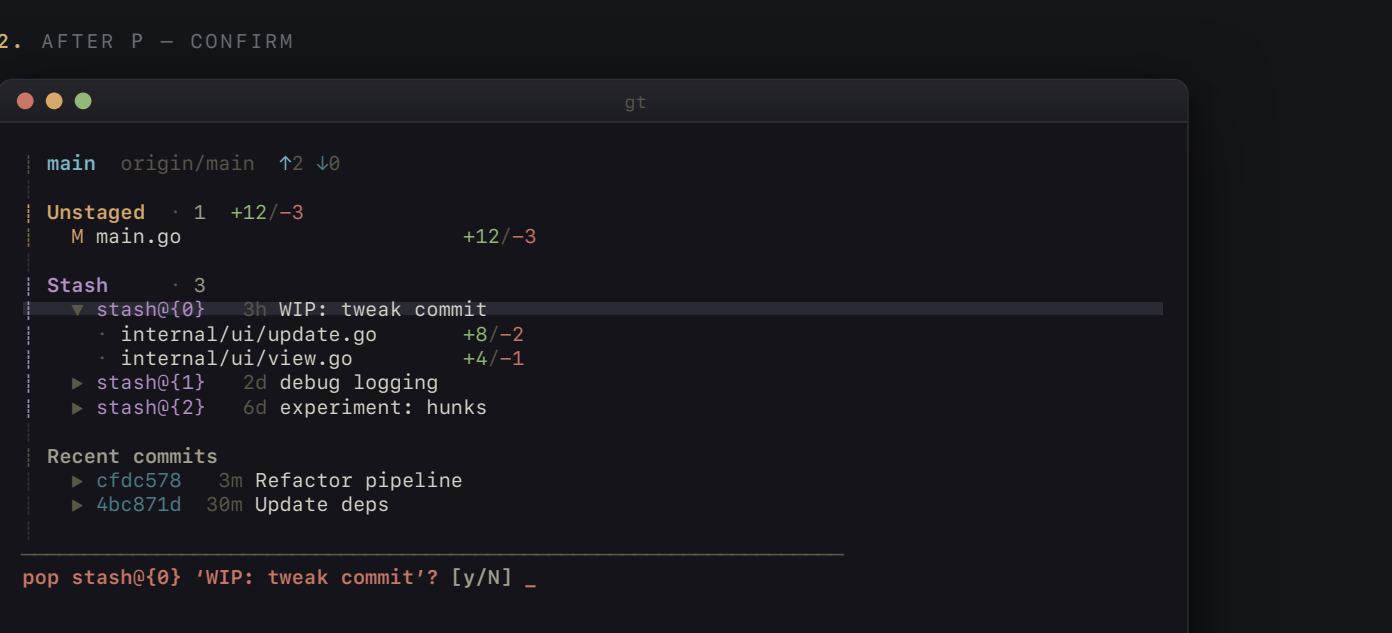
`l` on a stash row reveals its files (with diff stats). p pops — if conflicts are possible, gt asks first. The mauve rail dims as the stash drains.

1. STASH EXPANDED



```
gt

┆ main  origin/main  ↑2 ↓0

┆ Unstaged  · 1  +12/−3
┆   M main.go                      +12/−3

┆ Stash     · 3
┆   ▼ stash@{0}   3h WIP: tweak commit
┆       · internal/ui/update.go       +8/−2
┆       · internal/ui/view.go         +4/−1
┆   ▶ stash@{1}   2d debug logging
┆   ▶ stash@{2}   6d experiment: hunks

┆ Recent commits
┆   ▶ cfdc578   3m Refactor pipeline
┆   ▶ 4bc871d  30m Update deps
┆
──────────────────────────────────────────
d=diff p=pop a=apply h=collapse t=tag
```

2. AFTER P — CONFIRM



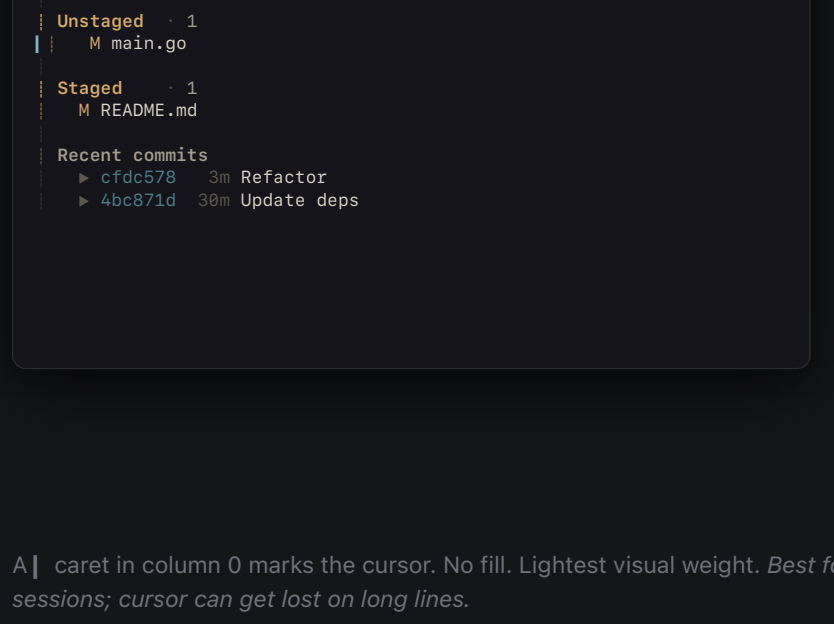
```
gt

┆ main  origin/main  ↑2 ↓0

┆ Unstaged  · 1  +12/−3
┆   M main.go                      +12/−3

┆ Stash     · 3
┆   ▼ stash@{0}   3h WIP: tweak commit
┆       · internal/ui/update.go       +8/−2
┆       · internal/ui/view.go         +4/−1
┆   ▶ stash@{1}   2d debug logging
┆   ▶ stash@{2}   6d experiment: hunks

┆ Recent commits
┆   ▶ cfdc578   3m Refactor pipeline
┆   ▶ 4bc871d  30m Update deps
┆
──────────────────────────────────────────
pop stash@{0} 'WIP: tweak commit'? [y/N] _
```

S capital because it's a destructive verb on the working tree — same convention as X for delete and A for amend. conflict on pop → toast & status bar prompt — g

Cursor & rail variants `EXPLORE`

Three options for the cursor row and the section attendance signal. My recommendation is the middle one (full-row dim fill + bright rail). The others are valid — pick what feels right at your terminal.

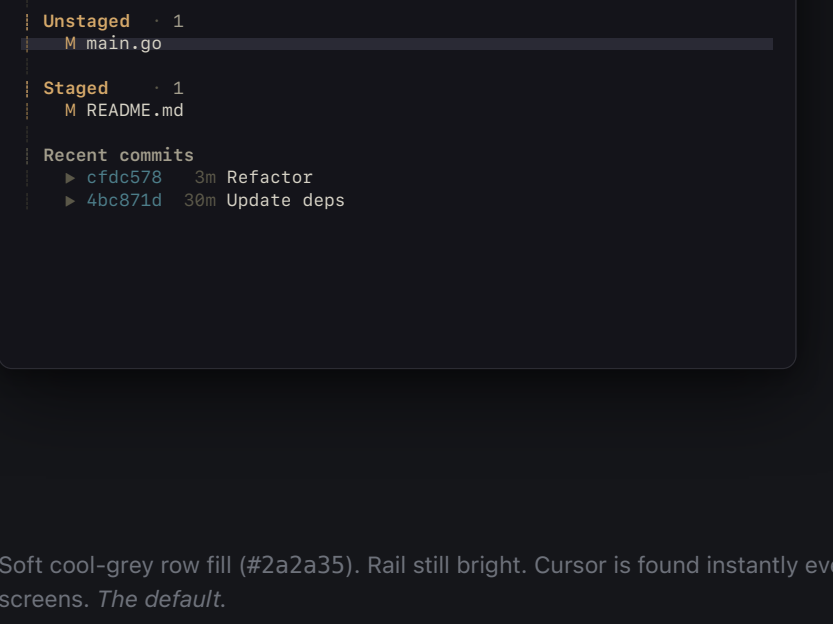
A · GUTTER CARET



```
gt — A

┆ main  origin/main  ↑2 ↓0

┆ Untracked  · 2
┆   ? .gitignore
┆   ? go.mod

┆ Unstaged   · 1
▎   M main.go

┆ Staged     · 1
┆   M README.md

┆ Recent commits
┆   ▶ cfdc578   3m Refactor
┆   ▶ 4bc871d  30m Update deps
```

A▎ caret in column 0 marks the cursor. No fill. Lightest visual weight. *Best for very dense sessions; cursor can get lost on long lines.*

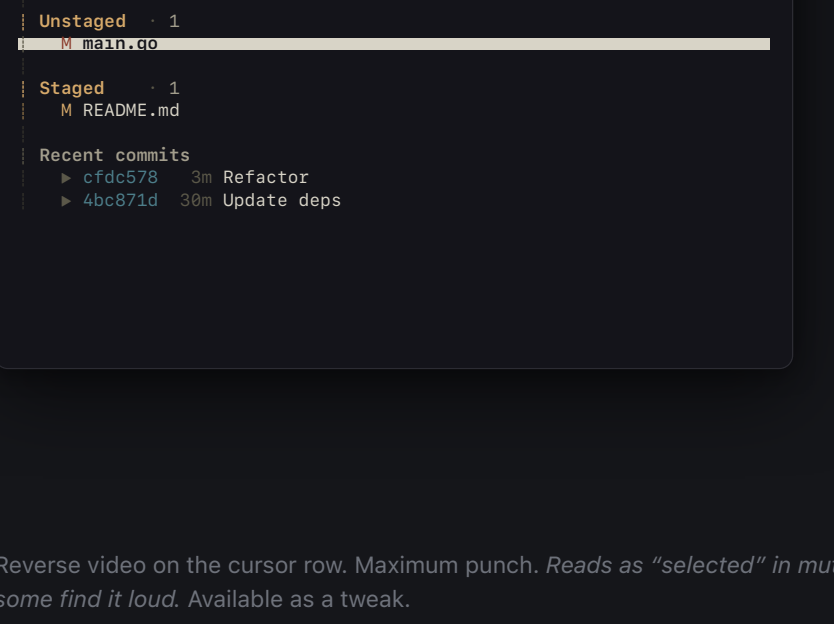
B · ROW FILL (RECOMMENDED)



```
gt — B

┆ main  origin/main  ↑2 ↓0

┆ Untracked  · 2
┆   ? .gitignore
┆   ? go.mod

┆ Unstaged   · 1
┆   M main.go

┆ Staged     · 1
┆   M README.md

┆ Recent commits
┆   ▶ cfdc578   3m Refactor
┆   ▶ 4bc871d  30m Update deps
```

Soft cool-grey row fill (#2a2a35). Rail still bright. Cursor is found instantly even on busy screens. *The default.*

C · INVERTED



```
gt — C

┆ main  origin/main  ↑2 ↓0

┆ Untracked  · 2
┆   ? .gitignore
┆   ? go.mod

┆ Unstaged   · 1
    M main.go

┆ Staged     · 1
┆   M README.md

┆ Recent commits
┆   ▶ cfdc578   3m Refactor
┆   ▶ 4bc871d  30m Update deps
```

Reverse video on the cursor row. Maximum punch. *Reads as "selected" in mutt & ranger; some find it loud.* Available as a tweak.

All three render correctly on truecolor terminals; B and C degrade to ANSI 241 / reverse on 16-color. c

Implementation notes `FOR CLAUDE CODE`

Sequenced so each step lands on its own. Each row maps to a small, reviewable PR. View paths are `internal/ui/view.go` unless noted.

STEP	CHANGE	FILES	SHIPS
01	Truecolor palette & named tokens. Replace `lipgloss.Color("3")` calls with named hex tokens.	view.go (top of file) — add `var theme = struct {...}`	v0.2 — visual refresh, no behavior change
02	Add **section rail:** prepend ┆ to every row. Bright on sections with files, ghost on empty sections.	view.go `renderRow` + `sectionHeader`	v0.2
03	Demote "Recent commits" to `styleSoft`. Make HEAD ref iris, not amber.	view.go `sectionHeader` + `renderRefs`	v0.2
04	Add **diff stat** to section headers and per-row right-align. Sum from `FileEntry.Added/Deleted`.	view.go `fileRow`, `sectionHeader`	v0.2
05	Cursor row pin: keep current full-row fill (B); expose `cursor.style` in a future config.	view.go `renderRow` — small	v0.2
06	**Inline diff (unified).** New mode `modeDiff`; parser for `git diff -U3 --no-color`; viewport with `[/]` hunk nav.	new: `internal/ui/diff.go`; wire from `update.go`'s `d` handler	v0.3 — *biggest change*
07	**Hunk-level staging.** `s` in diff mode → `git apply --cached` on a synthesized patch from one hunk.	diff.go + `internal/git/stage.go`	v0.3
08	**Split mode** (`v` toggle). Pair lines from the same parsed hunk; reuse stage plumbing.	diff.go — pure rendering	v0.3
09	**Stash section.** Add `git.SectionStash`; new `stash.go` wrapping `git stash list/show/pop/apply/drop/push -m`.	new: `internal/git/stash.go`; `model.go`, `view.go`	v0.4
10	**Stash keys:** `S` (new), `p` (pop), `a` (apply), `D` (drop). Reuse confirm + commit-prompt machinery.	update.go, keys.go	v0.4

> **Sequencing rationale.** 01–05 are pure rendering, no model changes — safe to ship as a polish PR. 06–08 introduce a new mode but reuse `tea.ExecProcess` only as a fallback (L). 09–10 add a section without changing the row model. Each ships independently.

> **Out of this doc, but worth a thought.** The diff-stat fetch is currently per-file via porcelain; consider `git diff --numstat` once and caching, so we can show stats on section headers without N+1 calls.